

Regular expression Denial of Service - ReDoS in [huggingface/transformers](#)

✓ Valid

Reported on Jun 14th 2025

Description

A regular expression denial of service (ReDoS) vulnerability has been identified in the Hugging Face Transformers library's MarianTokenizer. The vulnerability exists in the `remove_language_code()` method of the MarianTokenizer class, which processes text to remove language codes. The method can be exploited to cause excessive CPU consumption through crafted input strings containing malformed language code patterns.

Technical Details

- **Affected Component:**

```
transformers.models.marian.tokenization_marian.MarianTokenizer.remove_language_code
```

- **Vulnerability Type:** Regular Expression Denial of Service (ReDoS)
- **Attack Vector:** Maliciously crafted text input strings with language code patterns

The proof of concept demonstrates that by providing increasingly long strings with specific patterns of angle brackets and control characters, the execution time grows non-linearly, indicating potential catastrophic backtracking in the language code removal regex implementation.

Reproduction

Install the dependencies:

```
pip install transformers
pip install tensorflow
pip install sentencepiece
pip install sacremoses
```

Run the following code:

```
from transformers import MarianTokenizer
import time
```

```
model_name = 'Helsinki-NLP/opus-mt-en-fr'
tokenizer = MarianTokenizer.from_pretrained(model_name)

for i in range(1, 100000, 10000):
    payload = '<>>\x00' * i + '<\n>>><<'

    start = time.perf_counter()
    tokenizer.remove_language_code(payload)
    end = time.perf_counter()

    execution_time = end - start
    print(f'Payload length {i}, execution time: {execution_time:.4f} seconds')
```

Example output pattern:

```
Payload length 1, execution time: 0.0000 seconds
Payload length 10001, execution time: 0.3417 seconds
Payload length 20001, execution time: 1.3639 seconds
Payload length 30001, execution time: 3.1396 seconds
Payload length 40001, execution time: 5.5884 seconds
Payload length 50001, execution time: 8.5721 seconds
Payload length 60001, execution time: 12.3645 seconds
Payload length 70001, execution time: 16.7394 seconds
Payload length 80001, execution time: 22.3154 seconds
Payload length 90001, execution time: 27.8009 seconds
```

Impact

Example output shows increasing execution times with input length, demonstrating the vulnerability.

The provided proof of concept demonstrates the vulnerability by:

- Creating payloads of increasing length using repeated digit patterns
- Measuring execution time for number normalization
- Showing non-linear growth in processing time as input length increases

This vulnerability specifically affects language code preprocessing tasks using the `MarianTokenizer`'s `remove_language_code()` method, which has several implications:

- **Translation Preprocessing Disruption:**
 - The `remove_language_code()` method is used to strip language codes (e.g., `<en>`, `<fr>`) from input text before tokenization
 - Malicious input can cause excessive regex backtracking, delaying or stalling this step
 - Tokenization and translation tasks depending on this preprocessing step may fail or become unresponsive
- **Resource Exhaustion in Production:**

- Translation services using MarianTokenizer may experience high CPU utilization during input processing
- A single crafted request can block threads, degrading performance for concurrent users
- Translation job queues may grow uncontrollably or time out in batch environments
- **API Service Vulnerabilities:**
 - REST APIs or backend services that rely on MarianTokenizer for language-specific preprocessing are susceptible
 - Crafted payloads exploiting the regex logic in `remove_language_code()` can cause targeted denial of service
 - Traditional rate limiting may not be effective, as this is a CPU-bound issue triggered before request throttling can take effect
- **Development Impact:**
 - Developers using MarianTokenizer in testing or research workflows may face freezing or unresponsiveness
 - Preprocessing scripts for multilingual datasets may hang or fail unexpectedly during execution
 - Model training, evaluation, or inference pipelines relying on pre-tokenized data may be disrupted

Occurrences

 tokenization_marian.py L107

 We are processing your report and will contact the **huggingface/transformers** team within 24 hours. 7 months ago



 **Arjun Shibu** modified the report 7 months ago



 **Michelle Habonneau** validated this vulnerability 7 months ago

Hi, Thank you for taking the time to report. We've applied a fix in <https://github.com/huggingface/transformers/pull/39013> and will resolve this report when it's been released in the next version. Validating this report.

 **arjunshibu** has been awarded the disclosure bounty 

 The fix bounty is now up for grabs

 The researcher's credibility has increased: +7



CVE-2025-6638 assigned to this report. 7 months ago



Michelle Habonneau marked this as fixed in 4.53.0 with commit 47c34f 7 months ago

\$ The fix bounty has been dropped ❌

\$ tokenization_marian.py#L107 has been validated ✔️

✉ We have notified the **huggingface/transformers** maintainers about this report in their weekly follow-up 5 months ago

[Sign in](#) to join this conversation

CVE

CVE-2025-6638

Published

Vulnerability Type

CWE-1333: Inefficient Regular Expression Complexity

Severity

Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	Low

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

4.52.4

Visibility

Public

Status

Fixed

Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



Arjun Shibu

@arjunshibu

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)